

Stock Research Copilot

个人项目开发规格说明书 - 面向 Codex / AI Coding Agent 的实现文档

版本: v0.1 | 日期: 2026-05-06 | 目标: 从项目规划直接过渡到可执行代码任务

本文档用于保存项目设计，也可作为上传给 Codex、Cursor、Windsurf、GitHub Copilot 或其他代码生成工具的实现说明。文档重点是把需求拆成可编码的模块、接口、文件结构、验收标准和迭代任务。

项目属性	说明
项目类型	个人作品集项目 / 数据分析系统 / AI 辅助股票研究原型
第一版形态	Python + pandas + Streamlit Dashboard + Markdown/PDF 报告生成器
第一版市场范围	美股大盘股，建议先支持 AAPL, MSFT, NVDA, TSLA, AMZN, GOOGL, META, AMD, INTC, JPM
核心输出	结构化股票研究报告 + 财务指标 + 股价图表 + Bear/Base/Bull 估值区间
重要边界	仅用于教育和研究，不构成投资建议，不提供自动交易或个性化买卖建议

目录

1. 给 Codex 的最短使用说明
2. 项目目标与 MVP 范围
3. 数据源与合规边界
4. 系统架构与目录结构
5. 核心模块规格
6. 分析模型与估值逻辑
7. Streamlit Dashboard 页面设计
8. 自动报告生成规格
9. 开发里程碑与任务拆分
10. 测试、验收标准与质量要求
11. 给 Codex 的可复制主提示词
12. 附录: 配置、README、后续扩展

- 生成 Markdown 研究报告，并在 Streamlit 页面中展示。
- 提供基础同行对比，version one allows manual peers.yaml configuration.

2.4 非目标

- 不做自动交易，不接券商账户，不下单。
- 不输出确定性买入/卖出建议，不承诺收益。
- 不做高频数据，不处理 tick-level 数据。
- 不做复杂神经网络预测，除非 MVP 完成后作为扩展模块。
- 不解析付费研报全文，不抓取违反服务条款的数据。

3. 数据源与合规边界

第一版应尽量依赖公开、低门槛、可替换的数据源。价格数据可以使用 yfinance，它的文档说明该库提供从 Yahoo Finance 公开接口下载市场数据的 Python 方式，并声明适合研究和教育用途。SEC EDGAR API 可用于访问公司 submissions 和提取后的 XBRL company facts，这适合美国上市公司基本面分析。Streamlit 官方文档将其定义为面向数据科学和 AI/ML 工程师的开源 Python 动态数据应用框架。

数据类型	第一选择	替代/增强	备注
历史股价	yfinance	Alpha Vantage, Polygon, Tiingo	MVP 使用日线即可。
美国公司财报	SEC EDGAR company facts API	FMP, Finnhub	优先官方来源，缺点是字段映射复杂。
标准化财务报表	yfinance / FMP	SimFin, Intrinio	FMP 更结构化但可能需要 API key。
分析师目标价	FMP / Finnhub	手动录入或跳过	MVP 可作为可选模块。
新闻/公告	SEC filings, company IR	NewsAPI, Finnhub news	MVP 可以先不做实时新闻。

3.1 数据源引用

- SEC EDGAR APIs: <https://www.sec.gov/search-filings/edgar-application-programming-interfaces>
- Streamlit documentation: <https://docs.streamlit.io/>
- yfinance documentation: <https://ranaroussi.github.io/yfinance/>
- Financial Modeling Prep API docs: <https://site.financialmodelingprep.com/developer/docs>
- OpenAI Codex developer page: <https://developers.openai.com/codex>
- OpenAI Codex best practices: <https://developers.openai.com/codex/learn/best-practices>

3.2 合规与免责声明

所有页面、报告和 README 中必须出现免责声明: This project is for educational and research purposes only. It does not provide financial advice. The analysis is based on public data and model assumptions, which may be incomplete or inaccurate.

功能设计上应使用“情景估值”“合理区间”“风险提示”等表达，不使用“必涨”“稳赚”“立即买入”等确定性或诱导性表达。

4. 系统架构与目录结构

4.1 总体架构

User inputs ticker in Streamlit
|
v

```

Data Loader Layer
- price_loader.py      -> historical OHLCV
- financial_loader.py  -> income statement, balance sheet, cash flow
- sec_loader.py        -> official SEC facts, optional
  |
  v
Analysis Layer
- financial_ratios.py   -> growth, margins, leverage, returns
- technical_analysis.py -> MA, RSI, volatility, drawdown
- valuation.py         -> Bear/Base/Bull valuation
- peer_comparison.py   -> peer metrics table
  |
  v
Report Layer
- report_generator.py  -> Markdown research report
  |
  v
Presentation Layer
- streamlit_app.py     -> dashboard, charts, tables, report view

```

4.2 推荐目录结构

```

stock-research-copilot/
|
|-- README.md
|-- requirements.txt
|-- .env.example
|-- pyproject.toml          # optional, can be added later
|-- docs/
|   |-- stock_research_copilot_codex_spec.pdf
|-- data/
|   |-- raw/                # downloaded raw files, gitignored
|   |-- processed/          # processed csv/parquet/sqlite, gitignored
|   |-- reports/            # generated reports
|-- config/
|   |-- settings.yaml
|   |-- peers.yaml
|-- notebooks/
|   |-- 01_data_exploration.ipynb
|   |-- 02_financial_analysis.ipynb
|   |-- 03_valuation_model.ipynb
|-- src/
|   |-- __init__.py
|   |-- data_loader/
|   |   |-- __init__.py
|   |   |-- price_loader.py
|   |   |-- financial_loader.py
|   |   |-- sec_loader.py
|   |-- analysis/
|   |   |-- __init__.py
|   |   |-- financial_ratios.py
|   |   |-- technical_analysis.py
|   |   |-- valuation.py
|   |   |-- peer_comparison.py
|   |-- report/
|   |   |-- __init__.py
|   |   |-- report_generator.py
|   |   |-- templates/
|   |   |-- equity_report.md.j2
|   |-- app/
|   |   |-- __init__.py
|   |   |-- streamlit_app.py
|   |-- utils/
|   |   |-- __init__.py
|   |   |-- config.py
|-- tests/
|   |-- test_financial_ratios.py
|   |-- test_technical_analysis.py
|   |-- test_valuation.py

```

4.3 运行方式

```

# Install
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# Run dashboard

```

```
streamlit run src/app/streamlit_app.py

# Generate report from CLI, optional
python -m src.report.report_generator --ticker AAPL --out data/reports/AAPL_report.md

# Run tests
pytest -q
```

5. 核心模块规格

5.1 price_loader.py

函数	输入	输出	要求
get_price_history	ticker, period, interval	DataFrame[Date, Open, High, Low, Close, Adj Close, Volume]	使用 yfinance；处理空数据；统一日期索引。
get_company_profile	ticker	dict	包含 name, sector, industry, marketCap, currency 等基础信息。
validate_ticker	ticker	bool	ticker 为空或数据不可用时返回 False 或抛出自定义异常。

5.2 financial_loader.py

函数	输入	输出	要求
get_financial_statements	ticker	dict[str, DataFrame]	返回 income_statement, balance_sheet, cash_flow。
normalize_financials	raw dict	dict[str, DataFrame]	统一列名、日期方向和单位。
get_ttm_metrics	ticker	dict	尽量返回 EPS, revenue, net_income, free_cash_flow。

5.3 financial_ratios.py

函数	核心计算	输出字段
calculate_growth_rates	pct_change, CAGR	revenue_growth_yoy, net_income_growth_yoy, revenue_cagr
calculate_margins	利润 / 营收	gross_margin, operating_margin, net_margin, fcf_margin
calculate_returns	净利润 / 权益, NOPAT / invested capital	roe, roa, roic
calculate_leverage	债务/权益, 流动资产/流动负债	debt_to_equity, current_ratio, interest_coverage
generate_financial_summary	规则模板	中文/英文摘要字符串

5.4 technical_analysis.py

函数	计算逻辑	输出字段
add_moving_averages	rolling mean	MA20, MA50, MA200
calculate_rsi	14-day RSI	RSI

函数	计算逻辑	输出字段
calculate_volatility	$\text{daily return std} * \sqrt{252}$	annualized_volatility
calculate_max_drawdown	$\text{price} / \text{cumulative max} - 1$	max_drawdown
generate_technical_summary	规则模板	趋势、动量、风险解释

5.5 valuation.py

函数	方法	输出
estimate_by_pe	$\text{target_price} = \text{forward_eps} * \text{pe_multiple}$	Bear/Base/Bull price
estimate_by_ps	$\text{equity_value} = \text{revenue} * \text{ps_multiple};$ $\text{price} = \text{equity_value} / \text{shares}$	Bear/Base/Bull price
build_scenarios	合并默认假设和用户假设	ScenarioAssumptions dataclass
summarize_valuation	根据区间与当前价比较	估值解释文本

5.6 report_generator.py

第一版报告可以使用 Jinja2 模板生成 Markdown。后期可选用 WeasyPrint、ReportLab 或 Pandoc 导出 PDF。

函数	职责
build_report_context	将公司信息、财务指标、技术指标、估值结果整理成模板上下文。
render_markdown_report	填充 Markdown 模板，返回字符串。
save_report	将报告保存到 data/reports/{ticker}_report.md。
generate_report_for_ticker	一站式流程: data loading -> analysis -> valuation -> report。

6. 分析模型与估值逻辑

6.1 财务分析指标

指标	公式/逻辑	解释
Revenue Growth YoY	$(\text{Revenue}_t / \text{Revenue}_{t-1}) - 1$	收入增长能力。
Gross Margin	$\text{Gross Profit} / \text{Revenue}$	产品定价能力和成本结构。
Operating Margin	$\text{Operating Income} / \text{Revenue}$	主营经营利润率。
Net Margin	$\text{Net Income} / \text{Revenue}$	最终盈利能力。
Free Cash Flow	$\text{Operating Cash Flow} - \text{Capital Expenditure}$	现金创造能力。
FCF Margin	$\text{Free Cash Flow} / \text{Revenue}$	收入转化为自由现金流的能力。
ROE	$\text{Net Income} / \text{Shareholders Equity}$	股东资本回报效率。
Debt-to-Equity	$\text{Total Debt} / \text{Equity}$	财务杠杆风险。

6.2 技术分析指标

技术指标仅作为市场行为和风险提示，不替代基本面分析。报告中必须明确这一点。

指标	用途	基础判断规则
MA20/MA50/MA200	短中长期趋势	价格高于 MA200 表示长期趋势偏强；跌破则提示风险。
RSI	动量/过热	RSI > 70 偏过热；RSI < 30 偏超卖。
Volatility	年化波动率	波动率高说明价格不确定性更高。
Max Drawdown	历史最大回撤	用于理解极端下行风险。

6.3 情景估值模型

第一版不需要直接实现复杂 DCF。建议先实现 PE 和 P/S 两个相对估值模型。结果必须以区间和假设展示，不输出确定性价格预测。

```
PE valuation:
    target_price = forward_eps * pe_multiple

P/S valuation:
    equity_value = forward_revenue * ps_multiple
    target_price = equity_value / shares_outstanding

Scenario examples:
    Bear: lower EPS/revenue growth, lower valuation multiple
    Base: stable growth, valuation close to historical/peer average
    Bull: stronger growth, margin expansion, higher valuation multiple
```

情景	EPS 假设	PE 假设	解释
Bear	低于当前 TTM 或低增长	历史低位或同行低位	增长放缓、利润率承压、市场估值压缩。
Base	合理 forward EPS	历史均值或同行中位数	业务稳定，市场预期基本兑现。
Bull	高增长 EPS	历史高位或成长溢价	业绩超预期，利润率扩张，市场情绪积极。

6.4 规则化文字结论

```
Example rules:
- If revenue_growth_yoy > 0.15 and net_margin stable, describe growth as strong.
- If FCF margin is positive for 3 consecutive years, describe cash generation as healthy.
- If current PE is above 5-year average and revenue growth is slowing, flag valuation risk.
- If price is above MA200 but RSI > 70, describe long-term trend as strong but short-term overheated.
```

7. Streamlit Dashboard 页面设计

7.1 页面布局

```
Sidebar:
- Ticker input
- Period selector: 1Y / 3Y / 5Y / Max
- Peer tickers input
- Valuation method selector: PE / PS / blended
- Scenario assumptions expander

Main page:
1. Header: company name, ticker, sector, market cap
2. Price chart: close price + MA20/MA50/MA200
3. Key metrics cards: revenue growth, net margin, FCF margin, ROE, PE, PS
4. Financial charts: revenue, net income, free cash flow
5. Valuation table: Bear/Base/Bull
6. Peer comparison table
7. Auto-generated research report
8. Disclaimer and data source notes
```

7.2 页面验收标准

- 输入 AAPL 后页面在 10 秒内完成基础展示。
- 若数据源失败，页面显示友好错误，不崩溃。
- 所有图表标题、坐标轴和表格列名清晰。
- 报告区域可复制，且可以保存为 Markdown 文件。
- 页面底部显示数据源和免责声明。

8. 自动报告生成规格

8.1 报告模板

```
# {company_name} ({ticker}) Equity Research Report

## 1. One-line Summary
{one_line_summary}

## 2. Company Overview
{company_profile}

## 3. Price Trend
{price_analysis}

## 4. Financial Performance
{financial_analysis}

## 5. Valuation Analysis
{valuation_analysis}

## 6. Peer Comparison
{peer_comparison}

## 7. Bear / Base / Bull Scenarios
{scenario_table}

## 8. Key Risks
{risk_factors}

## 9. Watchlist
{watchlist}

## 10. Data Sources and Disclaimer
{sources_and_disclaimer}
```

8.2 报告质量要求

- 报告必须区分事实数据、模型假设和分析判断。
- 估值部分必须展示输入假设，而不是只展示结果。
- 风险因素必须至少包含基本面风险、估值风险、市场风险和数据质量风险。
- 报告中禁止出现确定性收益承诺。
- 报告必须包含生成时间、ticker、数据源和免责声明。

9. 开发里程碑与任务拆分

9.1 两周最小可运行版本

阶段	时间	任务	交付物
M0	Day 1	项目初始化，目录结构，requirements，README 初版。	可安装环境，pytest 能运行。
M1	Day 2-3	实现 price_loader，下载 AAPL 历史股价。	price DataFrame + 单元测试。
M2	Day 4-5	实现 technical_analysis，计算均线、RSI、波动率、回撤。	技术指标表 + 测试。
M3	Day 6-7	实现 financial_loader 和 financial_ratios。	关键财务指标 + 测试。

阶段	时间	任务	交付物
M4	Day 8-9	实现 valuation.py 的 PE/P/S 情景估值。	Bear/Base/Bull 表格。
M5	Day 10-11	实现 report_generator，输出 Markdown。	AAPL_report.md。
M6	Day 12-14	实现 Streamlit dashboard，整理 README 和截图。	可演示 MVP。

9.2 Codex 任务卡

任务卡	给 Codex 的目标	验收标准
T-01 Scaffold	创建项目结构、requirements.txt、README、.env.example、pytest 配置。	pytest -q 可运行；README 有安装与运行说明。
T-02 Price Loader	实现 yfinance 价格下载和公司 profile 查询。	对 AAPL 返回非空 DataFrame；异常可处理。
T-03 Technicals	实现 MA, RSI, volatility, max drawdown。	测试覆盖正常序列和空序列。
T-04 Financials	实现财务数据加载、标准化和关键指标计算。	能计算 revenue growth, net margin, FCF。
T-05 Valuation	实现 PE/P/S Bear/Base/Bull 情景估值。	结果为 DataFrame，并展示假设。
T-06 Report	实现 Markdown 报告生成。	生成 data/reports/AAPL_report.md。
T-07 Dashboard	实现 Streamlit 页面。	streamlit run 可启动并展示完整流程。
T-08 Tests & Docs	补充测试、类型提示、README 截图说明。	核心模块测试通过，README 可复现。

10. 测试、验收标准与质量要求

10.1 单元测试

```
tests/test_financial_ratios.py
- test_margin_calculation
- test_growth_rate_calculation
- test_free_cash_flow_calculation
- test_empty_financials_raise_clear_error

tests/test_technical_analysis.py
- test_moving_average_columns_exist
- test_rsi_range_between_0_and_100
- test_max_drawdown_is_non_positive

tests/test_valuation.py
- test_pe_valuation_formula
- test_ps_valuation_formula
- test_scenario_order_bear_base_bull
```

10.2 项目级验收

- 从空仓库按 README 安装后，能运行 Streamlit 页面。
- 默认 ticker=AAPL 能生成完整 dashboard 和 Markdown 报告。
- 测试通过: pytest -q。
- 无硬编码 API key，无隐私或凭据泄露。
- 关键函数有类型提示和 docstring。
- README 说明数据源限制和免责声明。

10.3 数据质量与错误处理

- 所有 loader 函数必须处理网络失败、空数据、ticker 不存在和字段缺失。
- 财务字段命名不一致时，应使用映射表和 fallback。
- 估值模型输入缺失时，返回“数据不足”而不是错误价格。
- 数据源结果应缓存到 data/raw 或 Streamlit cache，避免频繁请求。

11. 给 Codex 的可复制主提示词

以下提示词可以直接复制给 Codex。建议先让它只做 T-01 到 T-03，确认可运行后再继续。

You are implementing a personal portfolio project named Stock Research Copilot.
Read docs/stock_research_copilot_codex_spec.pdf as the product and engineering specification.
Build the project incrementally in Python.

Goal for the first implementation:

1. Create the repository structure described in the PDF.
2. Implement a minimal but working pipeline for ticker AAPL:
 - download historical price data with yfinance
 - calculate MA20, MA50, MA200, RSI, annualized volatility, max drawdown
 - load basic financial statements where available
 - calculate revenue growth, net margin, free cash flow and FCF margin where available
 - estimate Bear/Base/Bull valuation using PE and P/S assumptions
 - generate a Markdown equity research report
 - show everything in a Streamlit dashboard
3. Add pytest unit tests for ratio, technical indicator and valuation functions.
4. Add README.md with installation, usage, data source notes and disclaimer.

Constraints:

- Do not implement automatic trading.
- Do not provide financial advice or buy/sell recommendations.
- Do not hard-code API keys or secrets.
- The project must run without paid APIs in v0.1.
- Use type hints, docstrings, clear errors and modular code.
- After each task, run pytest -q and update README if needed.

Start by implementing only the scaffold, requirements.txt, config files,
price_loader.py, technical_analysis.py and their tests. Then stop and summarize what changed.

11.1 后续任务提示词示例

Task: Implement financial data loading and financial ratio analysis.

Requirements:

- Use yfinance first, and keep SEC EDGAR as an optional future loader.
- Implement get_financial_statements(ticker) and normalize_financials(raw).
- Implement calculate_margins, calculate_growth_rates, calculate_free_cash_flow.
- Add tests with small synthetic DataFrames so tests do not depend on network calls.
- Update README with the financial metrics supported.
- Run pytest -q.

Task: Implement valuation.py.

Requirements:

- Create dataclass ScenarioAssumption with fields scenario, forward_eps, pe_multiple, forward_revenue, ps_multiple, shares_outstanding.
- Implement estimate_by_pe, estimate_by_ps and blended_valuation.
- Return a pandas DataFrame with scenario, method, assumed_inputs and target_price.
- Add tests proving Bear <= Base <= Bull for normal inputs.
- Do not make buy/sell recommendations.

Task: Implement Streamlit dashboard.

Requirements:

- Sidebar ticker input, period selector and peer input.
- Main page sections: company overview, price chart, technical metrics, financial metrics, valuation scenarios, generated report and disclaimer.
- Use st.cache_data for data loading.
- Handle missing data gracefully.
- Keep the dashboard usable for AAPL, MSFT and NVDA.

12. 附录: 配置、README、后续扩展

12.1 settings.yaml 示例

```
default_ticker: AAPL
default_period: 5y
default_interval: 1d
cache_enabled: true
report_output_dir: data/reports
risk_free_rate: 0.04
valuation:
  pe:
    bear_multiple_discount: 0.75
    base_multiple: historical_average
    bull_multiple_premium: 1.25
  ps:
    bear_multiple_discount: 0.75
    base_multiple: peer_median
    bull_multiple_premium: 1.25
```

12.2 peers.yaml 示例

```
AAPL:
  - MSFT
  - GOOGL
  - META
  - AMZN
NVDA:
  - AMD
  - INTC
  - AVGO
  - QCOM
TSLA:
  - GM
  - F
  - RIVN
  - LCID
```

12.3 requirements.txt 建议

```
pandas
numpy
yfinance
streamlit
plotly
pyyaml
jinja2
python-dotenv
pytest
requests
```

12.4 README 核心内容

```
# Stock Research Copilot
```

A Python and Streamlit based equity research dashboard for educational use. It collects public market and financial data, computes financial ratios and technical indicators, builds Bear/Base/Bull valuation scenarios, and generates structured Markdown equity research reports.

```
## Quick Start
pip install -r requirements.txt
streamlit run src/app/streamlit_app.py
```

```
## Disclaimer
This project is for educational and research purposes only. It does not provide financial advice, investment recommendations, or trading signals.
```

12.5 进阶扩展路线

- 接入 SEC company facts API，构建更可靠的官方财务数据字段映射。
- 加入同行估值自动选择和行业分类。
- 加入 earnings call transcript / 10-K / 10-Q 的 RAG 问答。
- 加入新闻情绪分析，但必须保留来源引用。

- 加入 DCF 模型和敏感性分析矩阵。
- 加入 watchlist 和定时更新任务。
- 加入部署: Streamlit Community Cloud, Render, Railway 或 Docker。

12.6 最终简历描述

Stock Research Copilot | Python, pandas, Streamlit, yfinance, SEC EDGAR API

- Built an equity research dashboard that collects public market data and financial statements.
- Implemented financial ratio analysis, technical indicators, peer comparison and scenario-based valuation.
- Designed an automated Markdown report generator with explainable conclusions and data source notes.
- Added tests and a modular project structure suitable for further AI-assisted development.

结语

本项目的正确开发顺序是: 先让单只股票完整跑通, 再扩展到多只股票; 先做规则化可解释分析, 再引入 LLM; 先做研究报告和 Dashboard, 再考虑 App 化。

对于 Codex 等工具, 最有效的方式是把本 PDF 作为上下文, 并按照任务卡逐步实现。每次只提交一个明确任务, 要求它运行测试、解释改动、更新文档。